

N1X Cortex

Knowledge Management Methodology

Assisted by Artificial Intelligence

Turns large corpora of technical, regulatory, or strategic documentation into atomic knowledge graphs queryable by AI — with full traceability, verifiable compliance, and compliant code generation.

PREPARED BY

N1X Technologies

VERSION

2.0 · June 2026

STATUS

Published

Table of Contents

01 What is N1X Cortex

Definition, the problem it solves

02 Who it applies to

Domains: regulatory, technical, strategic, legal, operational

03 The 4 pillars

Atomize · Connect · Curate · AI Layer

04 Vault structure

Folders, frontmatter, wikilinks, MOCs

05 What it produces

Answers, compliance, compliant code, PDF documents

06 Document generation pipeline

matplotlib + Typst + attribution

07 Design principles

Fidelity, atomicity, citation, liberal linking

08 Application case

Hypothetical illustrative example

09 Authorship and IP

© 2026 N1X Technologies

01 — What is N1X Cortex

N1X Cortex is a methodology for turning large documentation corpora — technical, regulatory, strategic, legal, or operational — into **atomic knowledge graphs queryable by AI**, capable of generating precise answers, verifying compliance, and producing structured code or documents.

The problem

Monolithic documents do not scale for AI-assisted work.

A corpus of 50,000+ lines spread across dozens of files presents three fundamental problems:

Problem	Consequence
Context exceeds the AI window	The system cannot read the entire corpus at once; it generates answers without complete context
No semantic structure	The AI cannot reason about relationships between rules, flows, and errors
No traceability	There is no way to verify where each statement or decision comes from

The solution

N1X Cortex turns that documentary mass into a **network of atomic nodes**: one note per concept, one note per rule, one note per flow. All interconnected with semantic links and tagged with structured frontmatter.

- The result is a “second brain” that:
- Answers complex questions **citing the exact source**
 - Verifies whether a decision **complies with the domain’s rules**
 - Serves as precise context so that an AI can **generate compliant code**
 - Produces **structured documents** consolidating accumulated knowledge

02 — Who it applies to

N1X Cortex is applicable to any domain with **high documentary density and consistency requirements**:

Domain	Type of corpus	What it produces
Regulatory / fintech	Regulations, circulars, technical specifications	Verifiable compliance, code conforming to regulation
Legal / compliance	Contracts, policies, regulatory frameworks	Quick queries, identification of obligations
Strategic / product	Research, market analysis, roadmaps	Informed decisions, product documents
Technical / engineering	APIs, specs, architectures, runbooks	Code generated with the correct context
Operational	Processes, procedures, manuals	Quick consultation, workflow automation

When it is most valuable

The methodology is especially valuable when one or more of these criteria are met:

- The corpus exceeds **10,000 lines** and does not fit in a single AI context
- Rules and data **evolve frequently** (versions, regulatory updates)
- **Traceability** is needed: every answer must cite its source
- **Multiple people or systems** consult the same knowledge
- It is required to generate **code or documents** that reflect the domain’s rules

03 — The 4 pillars

01 Atomize

Split each source document into **minimal units of knowledge**. An atomic note contains exactly one idea: a concept, a rule, a flow, a message, an error code.

Atomicity criterion: if the note deals with two things that could change independently, it must be split into two.

Each note carries: YAML frontmatter with type, ID, tags, and status · structured body in natural language · implementation implications section (for flows and processes) · source citation at the foot with origin file and page.

02 Connect

Link each note to all related notes using **wikilinks** (`[[note-name]]`). Links are the connective tissue of the graph:

- A rule links to the flows that apply it
- A flow links to the messages it uses, the rules it complies with, and the possible errors
- A concept links to all the contexts where it appears

A link to a note that does not yet exist is **valid** — it marks future work without breaking the graph. This allows the system to be built incrementally.

03 Curate

Maintain the quality of the graph over time:

- **MOC (Maps of Content):** thematic indexes that group notes by domain
- **Glossary:** one note per domain term, with a canonical definition and aliases for equivalent terms
- **Duplicate merging:** when two notes represent the same concept, they are merged with aliases
- **Feedback loop:** every new learning returns to the brain as a new note or update

04 AI Layer

The curated graph becomes the **precise context** for AI systems:

- **Natural language queries:** the AI navigates the links and answers by citing the notes

- **Compliance verification:** the AI follows the link tree from a flow to its rules
- **Code generation:** the “Implementation implications” section of each note is the requirements checklist
- **Document generation:** strategic notes are consolidated into structured PDFs

04 — Vault structure

An N1X Cortex vault has the following generic structure, adaptable to any domain:

```
N1X-Cortex/  
├─ README.md      ← master plan and status  
├─ CLAUDE.md      ← conventions for AI  
├─ HOME.md        ← dashboard / entry point  
├─  
├─ 00-MOC/        ← thematic indexes  
├─ 01-Concepts/    ← atomic glossary  
├─ 02-Flows/       ← processes and flows  
├─ 03-Rules/       ← rules and obligations  
├─ 04-Technical/   ← APIs, messages, specs  
├─ 05-Errors/      ← error catalog  
├─ 06-Security/    ← security guidelines  
├─ 07-UX/          ← user experience  
├─ 08-MVPs/        ← code specifications  
├─ 09-Strategy/    ← analysis and opportunities  
├─  
├─ _templates/     ← note templates  
└─ Markdown/       ← original sources (DO NOT MODIFY)
```

Standard frontmatter

Each note carries minimal YAML frontmatter:

```
—  
type: flow      # concept | flow | rule | technical | strategy  
id: FLOW-EXAMPLE-01  
tags: flow, process, example  
source: "[[source-document-name]]"  
status: documented # draft | documented | verified  
—
```

05 — What it produces

A well-built N1X Cortex vault produces four types of outputs:

01 — Cited answers

Any question about the domain is answered with citations to the source notes. The AI enters through the topic's MOC, navigates the links, answers with the exact fact, and cites the note as the source.

02 — Compliance verification

For any flow or process, the AI navigates from the note to its linked rules and produces a verification list: compliant / non-compliant / pending.

03 — Compliant code

The “Implementation implications” section in each flow note lists the technical requirements. The AI uses that section + the links to messages, rules, and errors to generate code that already meets the domain's specifications.

04 — Structured documents

Strategic and analysis notes are consolidated into PDF documents through the generation pipeline described in Section 06 — with professional design, charts, and full attribution.

06 — Document generation pipeline

N1X Cortex includes a pipeline to convert accumulated knowledge into deliverable PDF documents — with consistent visual identity, authorship attribution, and traceability to the source notes. It is a reproducible method, not an ad-hoc design per document.

Guiding principle: the markdown is the source of truth; the PDF is a derived output. The PDF is never written by hand: it is compiled from a `.typ` file that reflects the markdown. If the content changes, change the markdown first and then regenerate.

Stack

Tool	What for	When
Typst (0.14+)	Typesetting engine → PDF. 10× faster than LaTeX, CSS-like syntax	By default
Python 3 + matplotlib	PNG charts to embed (≥180 DPI, brand palette)	If there are charts
HTML + headless Chrome	Alternative for highly visual designs (--print-to-pdf)	Optional

Generation flow

- 1 Consolidate** — The atomic notes from the thematic folder are gathered into structured markdown, preserving the source citations
- 2 Chart** (optional) — Generate the charts with Python/matplotlib (brand palette, ≥180 DPI, relative paths)
- 3 Lay out** — Write the `.typ`: color tokens, reusable components, cover, table of contents, sections, header and footer
- 4 Compile and verify** — `typst compile doc.typ doc.pdf` and review visually in Preview before publishing
- 5 Publish and version** — Copy the `.typ` and `.pdf` to the destination; keep the previous versions (`-v1`, `-v2...`), never overwrite

Anatomy, tokens, and components

Every deliverable follows the same structure: **cover** (title, version, date, “PREPARED BY”) → **table of contents** → **numbered sections** with consistent H1/H2/H3 headings → **footer** with automatic numbering → **final authorship footer**.

The palette is declared as tokens at the start of the .typ. **Brand separation rule:** a deliverable of the N1X Cortex methodology uses its own palette (navy/coral); a deliverable of a client project uses that client’s palette. They are never mixed. The reusable components (callout, coral-callout, hero-box, pillar-box, heading styles, and header/footer) are copied unmodified between documents of the same series.

Attribution

All generated documents carry author attribution in three places:

Place	Content
Cover	“PREPARED BY” field with the author’s full name
Footer of each page	Name · © · year
Final footer	Full authorship note with © and distribution restrictions

07 — Design principles

Principle	Description
Fidelity	Never invent data, rules, or steps. If the source does not say it, it is not asserted. Whatever requires verification is marked <code>status: draft</code> .
Atomicity	One idea per note. If a note grows until it covers two concepts that could change independently, it is split into two.
Mandatory citation	Every note cites its source at the foot: origin file and page number. Without a source, the note is not finished.
Liberal linking	Linking is done even if the destination note does not yet exist. An orphan link is a valid future-work marker, not an error.
Source immutability	The files in <code>Markdown/</code> are sacred — they are never modified. They are the immutable source of truth.
Living cycle	The brain is not built only once. Every new learning returns to the brain as a new note or update.
Incremental scale	It is not necessary to atomize the entire corpus before using the system. It expands on-demand, when needed.

08 — Application case (illustrative)

As a **hypothetical** example — it does not correspond to any real client or project — consider the application of N1X Cortex to an integration project in a regulated sector with high documentary density.

The corpus

Type	Volume	Content
Regulatory/technical	Tens of thousands of lines	API specifications, regulations, UX and cybersecurity guidelines, error catalogs
Functional workshops	A series	Documented process flows
Strategic	1 document	Research & Discovery: market, competitors, identified opportunities

The result

Several hundred atomic notes organized into thematic folders, with a graph of thousands of connections. It would allow answering complex questions about regulatory compliance, generating code that already meets the technical specifications, and producing structured strategic documents.

A case like this validates the methodology at its most demanding scale: a multidimensional corpus (technical + regulatory + strategic), multiple document versions, and the need for full traceability between each line of code and the rule that justifies it.

Note: The methodology is applicable to any project with similar characteristics — it is not specific to any sector or country.

09 — Authorship and IP

N1X Cortex is an original methodology developed by **N1X Technologies**.

The name, the four-pillar structure, the document generation pipeline, the atomization patterns, and the design components are the intellectual property of N1X Technologies.

The **applications of the methodology** are instances of use — the generated vault belongs to the context of each project, but the methodology itself belongs to its creator.

Prepared by **N1X Technologies** · N1X Cortex v2.0 · June 2026
© 2026 N1X Technologies — All rights reserved